

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_ARGS 64
#define BUF_SIZE 1024

// Global state tracking for previous directory [cite: 27]
char prev_dir[PATH_MAX] = "";

// Dispatch Table function pointer signature [cite: 27]
typedef int (*BuiltinHandler)(char **args);

typedef struct {
    const char *name;
    BuiltinHandler handler;
} BuiltinCommand;

// Forward declarations [cite: 27]
int builtin_cd(char **args);
int builtin_pwd(char **args);
int builtin_help(char **args);

// Dispatch Table for Built-in Commands [cite: 27]
BuiltinCommand dispatch_table[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"help", builtin_help},
    {NULL, NULL}
};

// 1. CD BUILT-IN HANDLER [cite: 27]
int builtin_cd(char **args) {
    char current_dir[PATH_MAX];
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd failed");
        return 1;
    }

    const char *target_dir = args[1];

    // Case A: No arguments passed (cd) -> default to $HOME [cite: 28]
    if (target_dir == NULL || strcmp(target_dir, "~") == 0) {
        target_dir = getenv("HOME");
        if (target_dir == NULL) {
            printf("cd: HOME variable not set\n");
        }
    }
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

```
// Dispatch Table for Built-in Commands[cite: 27]
BuiltinCommand dispatch_table[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"help", builtin_help},
    {NULL, NULL}
};

// 1. CD BUILT-IN HANDLER[cite: 27]
int builtin_cd(char **args) {
    char current_dir[PATH_MAX];
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd failed");
        return 1;
    }

    const char *target_dir = args[1];

    // Case A: No arguments passed (cd) -> default to $HOME[cite: 28]
    if (target_dir == NULL || strcmp(target_dir, "-") == 0) {
        target_dir = getenv("HOME");
        if (target_dir == NULL) {
            printf("cd: HOME variable not set\n");
            return 1;
        }
    }
    // Case B: Handle 'cd -' -> toggle to previous directory[cite: 28]
    else if (strcmp(target_dir, "-") == 0) {
        if (strlen(prev_dir) == 0) {
            printf("cd: OLDPWD not set\n");
            return 1;
        }
        target_dir = prev_dir;
        printf("%s\n", target_dir); // Print target when using cd -[cite: 28]
    }

    // Attempt directory change & handle errors[cite: 28]
    if (chdir(target_dir) != 0) {
        perror("cd failed");
        return 1;
    }

    // Update state: store previous directory and update PWD / OLDPWD[cite: 28]
    strncpy(prev_dir, current_dir, sizeof(prev_dir));
    setenv("OLDPWD", current_dir, 1);
    if (getcwd(current_dir, sizeof(current_dir)) != NULL) {
        setenv("PWD", current_dir, 1);
    }
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

```
        return 1;
    }
}
// Case B: Handle 'cd -' -> toggle to previous directory[cite: 28]
else if (strcmp(target_dir, "-") == 0) {
    if (strlen(prev_dir) == 0) {
        printf("cd: OLDPWD not set\n");
        return 1;
    }
    target_dir = prev_dir;
    printf("%s\n", target_dir); // Print target when using cd -[cite: 28]
}

// Attempt directory change & handle errors[cite: 28]
if (chdir(target_dir) != 0) {
    perror("cd failed");
    return 1;
}

// Update state: store previous directory and update PWD / OLDPWD[cite: 28]
strncpy(prev_dir, current_dir, sizeof(prev_dir));
setenv("OLDPWD", current_dir, 1);
if (getcwd(current_dir, sizeof(current_dir)) != NULL) {
    setenv("PWD", current_dir, 1);
}

return 0;
}

// 2. PWD BUILT-IN HANDLER[cite: 29]
int builtin_pwd(char **args) {
    char cwd[PATH_MAX];
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd failed");
        return 1;
    }
    return 0;
}

// Help menu[cite: 29]
int builtin_help(char **args) {
    printf("---- Directory Navigation Commands ----\n");
    printf("1. cd <path> : Change directory\n");
    printf("2. cd - : Change to previous directory\n");
    printf("3. cd ~ : Change to home directory\n");
    printf("4. pwd : Display current working directory\n");
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

~ - -zsh

~ - -zsh

~ - -zsh

~ - path_waitpid

~ - -zsh

UW PICO 5.09File: cd_dispatch.c

```
    }

    printf("\n[%s] > ", cwd_prompt);
    if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

    input_buf[strcspn(input_buf, "\n")] = 0;
    if (strcmp(input_buf, "exit") == 0) break;

    // Tokenize input[cite: 30]
    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(input_buf, " ");

    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " ");
    }
    args[arg_count] = NULL;

    // Execute via dispatch table[cite: 30]
    if (arg_count > 0) {
        int result = execute_builtin(args);
        if (result == -1) {
            printf("Error: '%s' is an invalid or non-builtin command.\n", args[0]);
        }
    }
}

return 0;
}
```

^G Get Help

^X Exit

^O WriteOut

^J Justify

^R Read File

^W Where is

^Y Prev Pg

^V Next Pg

^K Cut Text

^U UnCut Tex

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano cd_dispatch.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall cd_dispatch.c -o cd_dispatch
./cd_dispatch

[/Users/medikondarakesh] > pwd
/Users/medikondarakesh

[/Users/medikondarakesh] > cd /tmp

[/private/tmp] > cd -
/Users/medikondarakesh

[/Users/medikondarakesh] > cd /nonexistent_folder_123
cd failed: No such file or directory

[/Users/medikondarakesh] > exit
medikondarakesh@Medikondas-MacBook-Air ~ %
```